Puntatori

Description	Puntatori, Indirizzi di memoria, (**&** *****)

Puntatori

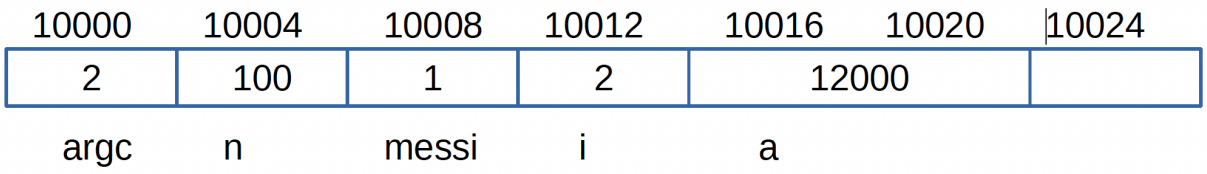
è una variabile che contiene l'indirizzo in memoria di un'altra variabile e si può indicare il suo tipo grazie al tipo della variabile puntata

```
int *a
    ^ Infichiamo che a è un indirizzo ed è di tipo int
```

I puntatori contengono tutti la stessa cosa (un indirizzo) ma il compilatore distingue tra

- int *
- long *
- double *

💡 Un'altro modo per generare puntatori è l'operatore **&**
*data una variabile w se scrivo **&w** ottengo l'indirizzo di **w** nella RAM*




```
&messi => 10008 // messi è nell'indirizzo di memoria 10008
&a => 10016 // a parte dall'indirizzo di memoria 10016
```

```
// variabile intera
int variabile;
```

```
// puntatore ad un intero
int *puntatore;
```

💡 L'asterisco (*) viene chiamato **operatore di indirezione o deferenziazione** e restituisce il contenuto dell'oggetto puntato dal puntatore;

mentre l'operatore (&) restituisce l'indirizzo di memoria della variabile

```
// assegno al puntatore l'indirizzo di variabile
puntatore = &variabile;
```

L’indirizzo di memoria può essere utile se vogliamo sapere il valore inserito in quella posizione

```
// metto in p l’indirizzo in RAM di messi
int *p = &messi;

// incremento n del valore della variabile a cui punta p (= messi)
n += *p;

// scrivo 7 nella variabile a cui punta p (= messi)
*p = 7;
```

💡 `*p = 7;` vuol dire che inserisco nella posizione di memoria ***p*** ma p conterrà sempre l’indirizzo di memoria iniziale

Gli operatori & e * servono per “aggirare” il fatto che nel C quando chiamo una funzione i parametri sono passati per valore

```
main() {
 ...
  int n = 7;
  fun(n);
 ...
}
```

```
int fun(int a) {
 ...
  x = a*a;
  a += x*y+a;
 ...
}
```

nulla di ciò che viene fatto in fun può. alterare il valore di n

```
main() {
 ...
  int n = 7;
  zun(&n);
 ...
}
```

```
int fun(int *p) {
 ...
  *p = 5;
 ...
}
```

Questa modalità va usata quando è strettamente necessario

Operazione	Descrizione
p + i	Sposta il puntatore di `i` posizioni in avanti
Equivale a:	`posizione + (i * dimensione del tipo puntato)`
p - i	Sposta il puntatore di `i` posizioni indietro
Equivale a:	`posizione - (i * dimensione del tipo puntato)`
p - q	È un intero che rappresenta il numero di elementi del tipo puntato che si possono inserire tra la posizione `p` e `q`
Equivale a:	`(posizione1 - posizione2) / dimensione del tipo puntato`

Ad esempio se il nostro puntatore `p`, punta alla prima posizione di un array di interi, `p++` punterà alla seconda posizione, avendo compiuto un balzo in avanti di 4byte, la dimensione dell'intero appunto.